

Processors and I/O

EMBEDDED — Lezione 4

Processor and Input and Output Devices

Other than the CPU, sono presenti molti **peripherals** (I/O devices): Timers, UART, Sensors.

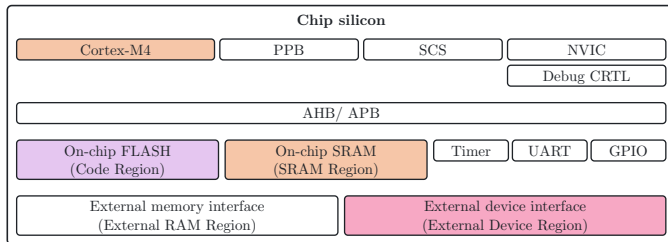


Fig. 1: On-chip FLASH, on-chip SRAM, peripheral region (Timer, UART, GPIO) e interfacce verso dispositivi esterni (External SRAM/FLASH, External LCD, SD card).

Il processore raggiunge gli **on-chip peripherals** attraverso la **bus matrix**; le **external interfaces** collegano dispositivi **off-chip** (External SRAM/FLASH, External LCD, SD card).

How Do They Communicate?

Ogni **peripheral** (device) espone dei **registers**. La CPU dialoga col dispositivo leggendo e scrivendo i **device registers**. Le interrupt lines (non mostrate nella figura) riportano eventi dai dispositivi I/O che richiedono servizio della CPU.



Fig. 2: CPU collegata ai registri Status e Data; i registri sono collegati al Device Mechanism.

Device registers link software e hardware state

I device registers collegano il software allo stato hardware.

Data registers Carry values moving into or out of the device.

- hold data values
- access direction depends on the peripheral
- can be readable or writable

Status and control registers Report events and configure operating modes (e.g., event: the current operation has been completed).

- individual bits often have different meanings

Two architectural models for device access

Due modi per accedere ai registri.

- **I/O instructions:** istruzioni speciali per input/output; in e out nel caso dell'Intel x86; forniscono un **separate address space** per i dispositivi I/O (separate address lines...).
- **Memory-mapped I/O:** fornisce indirizzi per i registri di ciascun dispositivo I/O; i programmi usano le normali istruzioni di memory read/write della CPU per comunicare coi dispositivi.

Model	Addressing	Instructions	Typical example
Separate I/O space	Dedicated device address space	Special input and output instructions	Legacy x86 ports
Memory-mapped I/O	Device registers share the memory address space	Normal load and store instructions	Arm microcontrollers

Device register access in ARM and C

ARM loads and stores access device registers

Definire la location del device e fornire il codice Read/Write:

```
DEV1_BASE EQU 0x40001000
LDR r1, = DEV1_BASE
LDR r0, [r1] // read register
MOVS r0, #8
STR r0, [r1] // write register
```

What matters L'address deve identificare il registro corretto; access width e alignment devono corrispondere alla specifica del peripheral. Un address errato può controllare un altro peripheral o causare un **bus fault**.

C requires volatile access to device registers

Si possono usare i pointer per manipolare gli indirizzi dei dispositivi I/O.

```
#include <stdint.h>
#define DEV1_STATUS_ADDR 0x40001000u
#define DEV1_STATUS (*(volatile uint32_t *)DEV1_STATUS_ADDR)
```

```
uint32_t status = DEV1_STATUS;
DEV1_STATUS = 8u;
volatile indica al compiler che ogni read e write conta, perché l'hardware può cambiare il valore fuori dal normale flusso del programma.
```

Introducendo funzioni che leggono e scrivono memory locations:

```
#define DEV1_STATUS_ADDR 0x40001000u
```

```
int read(uint8_t *location) {
    return *location;
}
```

```
void write(uint8_t *location, char newval) {
    (*location) = newval;
}
```

```
uint32_t status = read(DEV1_STATUS_ADDR); /* read device register */
write(DEV1_STATUS_ADDR, 8); /* write 8 to device register */
```

Register definitions should express width and intent

Il device reference manual definisce il legal access pattern per ogni device register.

Concern	Reason
Access width	A byte, halfword, or word transfer may have different hardware meaning
Read-only and write-only behavior	Some registers reject unsupported access directions
Reserved bits	Software should preserve or write required values
Side effects	A read may clear status and a write may acknowledge an event

Polling (Busy-Wait) I/O

- I dispositivi I/O sono tipicamente più lenti della CPU: possono richiedere molti cicli per completare un'operazione.
- **Busy-wait** (spesso chiamato **polling**): chiedere a un dispositivo I/O se ha finito leggendo il suo **status register**; il software legge lo status register finché non appare la condizione **ready** o **complete**.
- Il loop fornisce bassa complessità di control-flow, ma i cicli CPU vengono consumati mentre non progredisce lavoro utile.

Polling an output device (Example)

- Un carattere avvia ogni transazione.
- Il loop interno attende il completamento.

```
#define OUT_CHAR 0x1000 /* output device character register */
#define OUT_STATUS 0x1001 /* output device status register */
```

```
while (*current != '\0') {
    OUT_DATA = *current;
    OUT_STATUS = START;

    while (OUT_STATUS != DONE) {
        // CPU waits here
    }

    current++;
}
```

La CPU non può svolgere lavoro applicativo indipendente durante l'attesa.

A polling-based input-to-output loop (Example)

Ripetere per sempre:

- attendere che l'input status riporti dati disponibili;
- leggere un valore dal input data register;
- scrivere quel valore nel output data register;
- attendere che il output device completi la transazione.

```
for (;;) {
    while (IN_STATUS == EMPTY) { }
    uint8_t c = IN_DATA;

    while (OUT_STATUS == BUSY) { }
    OUT_DATA = c;
    OUT_STATUS = START;
}
```

Il loop non permette al foreground work di continuare.

Interrupt-driven I/O

- Il busy-wait I/O è inefficiente: la CPU potrebbe fare lavoro utile in parallelo all'I/O.
- Il meccanismo degli interrupt permette a un dispositivo di richiedere servizio solo quando si verifica un evento.
- Il processore salva il context, esegue un handler, poi riprende il programma interrotto.

Several signals control the interrupt process

- Il dispositivo asserisce un **interrupt request**: la logica del dispositivo I/O decide quando interrompere (e.g., quando i dati sono pronti).
- La CPU accetta la richiesta: asserisce il segnale **interrupt acknowledge** e cambia il program counter in modo che punti all'**interrupt handler** del dispositivo.
- L'**interrupt handler** identifica la sorgente dell'evento (e.g., quale dispositivo) e compie le azioni di processing necessarie.

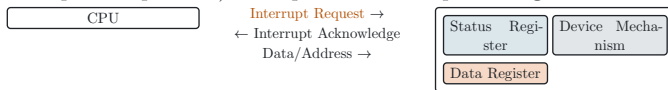


Fig. 3: Interrupt request, interrupt acknowledge e data/address tra CPU e dispositivo.

Interrupt entry changes control flow

Quando avviene un interrupt:

- per prima cosa viene salvato il valore del PC al momento dell'interruzione, così la CPU può tornare al foreground program;
- poi il PC punta a una interrupt handler routine, che serve il dispositivo leggendo i dati appena pronti, scrivendo i successivi dati...

Phase	Processor action
Recognition	Detect interrupt
Context save	Preserve the return state and selected registers
Vector fetch	Load the handler address from the vector table
Service	Execute the interrupt handler
Return	Restore state and resume the interrupted code

Cortex-M interrupt entry

- Salva il context necessario: pushes dei registri importanti R0-R3, R12, LR, PC e xPSR nello stack.
- Trova l'handler address dall'interrupt vector table; l'handler può salvare registri aggiuntivi che usa.
- L'interrupt return ripristina lo stato e il context salvati dall'hardware.
- Il codice interrupt in C è processato dal compiler in modo specifico: gli handler generati dal compiler seguono l'architecture calling convention quando dichiarati correttamente.

A minimal input interrupt handler in C

- Legge il dato che ha causato l'interrupt.
- Registra lo stato minimo per il processing successivo.
- Acknowledge o clear dell'evento del dispositivo.
- Ritorna rapidamente.

```
volatile uint8_t latest_char;
volatile bool char_ready;
```

```
void input_handler(void) {
    latest_char = IN_DATA; // read device register
    char_ready = true;
    IN_STATUS = ACK; // acknowledge the device
}
```

Interrupts example

Copiare caratteri da input a output con interrupt di base:

- achar: passa il carattere al foreground program;
- gotchar: segnala quando è stato ricevuto un nuovo carattere.

```
/* INTERRUPT HANDLERS */
/* get a character and put in global (called when IN_STATUS is 1) */
void input_handler() {
    achar = read(IN_DATA); /* get character */
    gotchar = TRUE; /* signal to main program */
    write(IN_STATUS, 0); /* reset status to initiate next transfer */
}
```

```
/* react to character being sent (called when OUT_STATUS is 0)
```

```
*/
void output_handler() {
    /* don't have to do anything */
}
/* The main program is a bit simpler than Busy Wait I/O,
but it still doesn't allow the foreground program to do useful work:
it just polls for input and writes data.
main() {
    while (TRUE) { /* read then write forever */
        if (gotchar) { /* write a character */
            write(OUT_DATA, achar); /* put character in device */
            write(OUT_STATUS, 1); /* set status to initiate write */
            gotchar = FALSE; /* reset flag */
        }
    }
}
```

A circular buffer separates producer and consumer timing

- Un indice identifica la next read position.
- L'altro identifica la next write position.



Fig. 4: Buffer circolare a 8 celle; un indice per la next read position, l'altro per la next write position.

Buffer implementation

```
#define BUF_SIZE 8
char io_buf[BUF_SIZE]; /* character buffer */
int buf_head = 0, buf_tail = 0; /* current position in buffer */
int error = 0; /* set to 1 if buffer ever overflows */

int buffer_empty() { /* returns TRUE if buffer is empty */
    return buf_head == buf_tail;
}

int buffer_full() { /* returns TRUE if buffer is full */
    return (buf_tail+1) % BUF_SIZE == buf_head;
}

int nchars() { /* returns the number of characters in the buffer */
    if (buf_head >= buf_tail)
        return buf_head - buf_tail;
    else
        return BUF_SIZE - buf_tail - buf_head;
}

void buffer_put(char achar) { /* add a character to the buffer head */
    io_buf[buf_tail++] = achar;
    /* check pointer */
    if (buf_tail == BUF_SIZE)
        buf_tail = 0;
}

char buffer_get() { /* take a character from the buffer head */
    char achar;
    achar = io_buf[buf_head++];
    /* check pointer */
    if (buf_head == BUF_SIZE)
        buf_head = 0;

    return achar;
}
```

Interrupts example with circular buffer

Input ISR adds data and starts output

- Cattura l'elemento in arrivo.
- Registra l'overflow senza bloccarsi.
- Avvia l'output solo quando la coda era idle.
- Acknowledge dell'evento di input.

```
void input_handler(void) {
    uint8_t c = IN_DATA;

    if (buffer_full()) {
        overflow = true;
    } else {
        bool was_empty = buffer_empty();
        buffer_put(c);
        if (was_empty) start_output();
    }

    IN_STATUS = ACK;
}
```

Output ISR launches the next queued item

- Ogni completion interrupt lancia il prossimo elemento in coda.

- L'handler disabilita gli interrupt non necessari quando non restano dati.

```
void output_handler(void) {
    OUT_STATUS = ACK;
    if (!buffer_empty()) {
        OUT_DATA = buffer_get();
        OUT_CONTROL = START;
    } else {
        stop_output();
    }
}
```

Foreground vs interrupt

Il foreground program viene occasionalmente interrotto da operazioni di input e output, gestite dagli interrupt handler in background; il foreground program riprende dopo gli handler.

```
main() --Input ISR--> main() --Output ISR--> main() --Input
ISR--> main()
           (background)           (background)
(background) --> Time
```

Bugs

- Gli errori possono essere molto difficili da trovare quando gli interrupt handler hanno bug, a causa della concurrency.
- L'interrupt handler deve salvare ogni registro CPU che modificherà e ripristinarli prima di uscire.
- Dimenticare di salvare/ripristinare un registro nell'handler può far cambiare misteriosamente una variabile del foreground program.

Read-modify-write race

Esempio: impostare bit[3] nel word data all'address 0x20000000.

```
;Read-Modify-Write Operation
LDR R1, =0x20000000 ;Setup address      LDR R1,
=0x20000000 ;Setup address
LDR R0, [R1]        ;Read 0x21          LDR R0, [R1]        ;Read
0x21
...                  Interrupt!          ORR.W R0, #0x8        ;Set
bit[3]
LDR R1, =0x20000000 ;Setup address      STR R0,
[R1]                ;Write back 0x29
STR R2, [R1]        ;Write back 0x79    ...
...                  Return!
```

Read-modify-write operation

- Legge il dato (0x21) dall'address 0x20000000.
- L'interrupt cambia il dato dell'address 0x20000000 e riscrive il vecchio dato modificato.
- 0x79 è andato perso!

Main program: Read data --> Modify bit[3] ----->
Write data back

```
ISR:                Interrupt occurs
                    Read data -> Modify bit[3] ->
Write data back
```

(Interrupt Service Routine)

Bit[3] modificato dall'ISR viene sovrascritto dal main program.

Interrupt handlers should do bounded work

- Catturare o consegnare il minimo dato richiesto dal dispositivo.
- Clear della sorgente di interrupt prima che possa riattivarsi inaspettatamente.
- Evitare blocking calls e loop non limitati.
- Spostare il processing costoso nel foreground o in deferred work.
- Misurare il worst-case execution time.

Interrupts - Implementaton

- La CPU controlla la **interrupt request (IRQ)** line a ogni istruzione.
- Se una interrupt request è stata asserita, la CPU:
 - mette il return address su uno stack (come fa per le subrou-tine);
 - non fetcha l'istruzione puntata dal PC, ma setta il PC all'inizio dell'**interrupt handler**.
- Gli address degli interrupt handler sono memorizzati in una tabella.

Priorities and Vectors

- La maggior parte dei sistemi ha più di un dispositivo I/O: più dispositivi possono interrompere; più interrupt handler, device address.
- **Interrupt priorities**: riconoscere alcuni interrupt come più importanti di altri.
- **Interrupt vectors**: permettere al dispositivo che interrompe di specificare il proprio interrupt handler.

Programmable interrupt controller (PIC)

- Il **Programmable interrupt controller (PIC)** priorizza più sorgenti di interrupt in modo che, in ogni momento, l'interrupt a priorità più alta sia presentato alla core CPU per il processing.
- Cortex-M integra questa funzione nell'**NVIC**.

```
Airbag Sensor HIGHEST \
Break Sensor HIGH \
Real Time Clk MED > PIC --> CPU
Fuel Level LOW / (Interrupt Vector)
```

Fig. 1: Sorgenti di interrupt con priorità HIGHEST/HIGH/MED/LOW verso il PIC, poi verso la CPU; il PIC fornisce l'Interrupt Vector.

Priority logic and nested interrupts

- La **priority logic** seleziona la highest eligible request; il PIC memorizza il priority level di quell'interrupt in un registro interno.
- Quando sopraggiunge un interrupt successivo, la priorità è confrontata con la priorità corrente.
- **Nested interrupts**: una sorgente a priorità più alta può pre-emptare il processing di un interrupt a priorità più bassa.

Pending bits, masking, NMI

- I **pending bits** registrano gli interrupt in attesa di servizio.
- **Masking interrupts**: gli enable bits determinano quali sorgenti possono interrompere.
- L'interrupt a priorità più alta è chiamato **nonmaskable interrupt (NMI)**.
- L'NMI non può essere disattivato: e.g., di solito riservato a interrupt causati da power failures, per salvare critical state in memoria non volatile, spegnere dispositivi I/O...

Interrupt Vector Table

- Ogni interrupt ha un **vector number**, che indicizza la vector table.
- La entry selezionata contiene l'**interrupt vector**, cioè il memory address dell'interrupt handler.

Interrupt Vector Table

Handler 1	Vector 0
Handler 3	Vector 1
Handler 4	Vector 2
Handler 2	Vector 3

Overhead of Interrupts

Un interrupt causa un cambiamento del program counter (incombe in un branch penalty):

- l'interrupt può salvare automaticamente alcuni registri CPU, richiedendo cicli extra;
- acknowledge dell'interrupt e ottenimento dell'interrupt vector richiedono cicli extra;
- l'overhead dell'interrupt handler: salva e ripristina i registri CPU non salvati automaticamente; la interrupt return instruction ripristina lo stato salvato automaticamente e incorre in un branch penalty.

Interrupt Response Time Il tempo richiesto dall'hardware per rispondere all'interrupt, ottenere il vector, salvare lo stato e così via, **non può essere cambiato dal programmatore**.

